

Claude Code In One Shot - TrainWithShubham

The Complete Guide. Beginner to power user. Every feature, every file, every hack. Companion reference for the YouTube video **"Claude Code in One Shot"** on **TrainWithShubham**. **2026 Edition** — Routines, Checkpoints, scoped hooks, Agent Teams, 5 surfaces.

```
CLAUDE CODE IN ONE SHOT · TRAINWITHSHUBHAM · 2026
Web:      trainwithshubham.com
YouTube:  @TrainWithShubham
Author:   Shubham Londhe
License:  free to share, attribute when you remix
```

 **Site** — trainwithshubham.com  **Watch the video** — youtube.com/@TrainWithShubham 
Follow Shubham — [@trainwithshubham](https://twitter.com/trainwithshubham)  **Community** — DevOps + AI engineers shipping with Claude Code

If this resource saves you a week, the way to pay it back is: **subscribe, share with one teammate, and ship something with Claude Code this weekend.** That's it.

Table of Contents

- 0. Cold Open — What is Claude Code
- 1. Install & First Run
- 2. The Core Loop
- 3. Permission Modes — the safety dial
- 4. `CLAUDE.md` — memory you write
- 5. Auto Memory — memory Claude writes
- 6. The `.claude/` folder
- 7. `settings.json` — the control panel
- 8. Slash Commands — built-in + custom
- 9. Skills — packaged know-how
- 10. Subagents & Agent Teams
- 11. Hooks — deterministic automation

- |2. MCP — connect anything
 - |3. Plugins & Marketplaces
 - |4. IDE Integrations & the 5 Surfaces
 - |5. Headless Mode + Agent SDK
 - |6. Routines — cloud-scheduled agents
 - |7. Checkpoints — undo for AI
 - |8. Output Styles
 - |9. Hidden Gems & Power-User Hacks
 - |10. Pro Patterns — combining everything
 - |11. Troubleshooting
 - |12. Cheat Sheet
-

0. Cold Open

Claude Code is a coding agent that runs in your terminal, your IDE, your browser, your phone, and on Anthropic's cloud while your laptop is closed. Not a chat window. Not a copilot. A full agent — it reads files, runs commands, edits code, opens browsers, queries databases, schedules itself, and ships features.

Four things make it different from every other AI coding tool:

1. **It runs in your shell.** Not a sandbox. It sees your real repo, your real env vars, your real `node_modules`.
2. **It's extensible at every layer.** Skills, subagents, hooks, MCP, plugins, routines — everything is a markdown file or a JSON config.
3. **It composes.** A skill calls a subagent. A hook blocks a tool. An MCP server exposes tools. A routine fires the whole stack on a schedule.
4. **It runs everywhere.** Terminal, VS Code, JetBrains, Desktop app, Web (`claude.ai/code`), and mobile via the Claude iOS app. Same `claude` CLI under the hood.

By the end of this video, you'll have built your own. Let's go.

1. Install & First Run

Install

```
# macOS / Linux / WSL – recommended (auto-updates)
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# Homebrew
brew install --cask claude-code

# npm
npm install -g @anthropic-ai/claude-code

# Pin a specific version (or "stable" / "latest")
claude install latest
curl -fsSL https://claude.ai/install.sh | bash -s 2.1.126
```

The 5 surfaces

Surface	How to launch	When to use
Terminal	<code>claude</code>	Default. Power-user mode.
VS Code	extension <code>Anthropic.claude-code</code>	IDE diff view + selection context
JetBrains	Marketplace plugin “Claude Code”	IntelliJ/PyCharm/WebStorm/etc.
Desktop	<code>/desktop</code> from terminal, or download app	Multi-session UI, drag-drop files
Web	<code>claude.ai/code</code>	Zero-install, runs in browser sandbox

Plus the **Claude mobile app** (`/mobile` shows a QR) for **Remote Control** — kick a session running on your laptop from your phone.

First Run

```
cd ~/your-project
claude          # opens browser → /login → done
```

That's it. Type a prompt, hit enter.

Verify install

```
claude --version
claude doctor      # diagnostic – run this when anything breaks
claude update      # update manually (Homebrew users use brew upgrade)
```

Pull a session across surfaces

```
/teleport          # pull a Claude Code web session into THIS terminal (alias /tp)
/desktop           # hand current session off to the Desktop app (alias /app)
/web-setup         # connect your GitHub to claude.ai/code via local gh CLI
```

Gotcha

- Native installer auto-updates. **Homebrew does not** — `brew upgrade claude-code`. Use `claude install latest` to pin the latest channel from the binary itself.
 - On Windows, install **Git Bash** if you want bash semantics, or accept the new PowerShell tool (auto-rolling out as of v2.1.111). Set `CLAUDE_CODE_USE_POWERSHELL_TOOL=0` to opt out.
 - The Web surface runs in a sandbox — no access to your local files unless the cloud environment mounts the repo. Routines (§16) run there too.
-

2. The Core Loop

Once inside `claude`, the loop is:

1. **You type a prompt** (or paste a task, drop a file).
2. **Claude reads, thinks, calls tools** (Read, Edit, Bash, Glob, Grep, WebFetch...).
3. **You watch tool calls scroll by**, approving anything that needs permission.
4. **You correct, redirect, or ship.**

The keys you need on day one

Key	Action
<code>Enter</code>	Send prompt
<code>Shift+Enter</code>	Newline in prompt
<code>Esc</code>	Interrupt Claude (stop a tool / response)
<code>Esc Esc</code>	Open Checkpoint menu — restore code+conversation, conversation only, code only, or summarize-from-here (see §17)
<code>Shift+Tab</code>	Cycle permission modes (default → acceptEdits → plan → auto)
<code>Ctrl+B</code>	Send running task to background
<code>Ctrl+O</code>	Toggle verbose transcript (see raw tool I/O)
<code>Ctrl+U</code>	Clear input buffer
<code>Ctrl+L</code>	Redraw screen
<code>Ctrl+C</code>	Quit
<code>!</code> (start of line)	Run as bash directly, capture output
<code>#</code> (start of line)	Save line to <code>CLAUDE.md</code> memory
<code>@filename</code>	Inject a file into the prompt
<code>/</code>	Slash-command menu

Special prefixes — the trick most people miss

```
! npm test           # run bash, output added to Claude's context
# always use 2-space indent # appended to CLAUDE.md immediately
@src/auth.ts is broken # injects file content + asks question
```

A worked example — your first 60 seconds

```
> @src/auth.ts the login flow drops the refresh token after 5 min, why?  
  
[Claude reads auth.ts, runs `grep -n refresh src/`, opens 2 more files]  
[Proposes a 4-line edit]  
[You hit Shift+Tab → acceptEdits, then Enter]  
[Edit lands, prettier hook auto-formats, tests run]  
  
> #always set token TTL from env, never hardcode  
[Saved to CLAUDE.md]  
  
> ship this  
[Claude invokes /commit skill, writes a conventional commit, pushes]
```

That's the whole job. Everything else in this guide is making this loop faster, safer, and reproducible across your team.

3. Permission Modes

Claude asks before doing risky things. The mode controls "risky."

Mode	Auto-approves	Use it for
<code>default</code>	Reads only	Starting cold, sensitive repos
<code>acceptEdits</code>	Reads + Edits + safe bash (<code>mkdir</code> , <code>mv</code> , <code>cp</code> , <code>sed</code>)	Active iteration on code
<code>plan</code>	Reads only — drafts a plan, asks before executing	Big refactors, "what would you do"
<code>auto</code>	Classifier-driven — auto-approves safe, escalates risky (Max-only on Opus 4.7)	Long-running autonomous sessions
<code>dontAsk</code>	Suppresses prompts entirely — denies what isn't pre-approved instead of asking	Headless / piped runs that mustn't block on prompts
<code>bypassPermissions</code>	Everything. No checks.	Disposable VMs / containers only

Inspect the auto-mode classifier and recent denials with:

```
claude auto-mode          # view classifier config
/permissions              # auto-mode denials are reviewable here
```

Cycle modes live with `Shift+Tab` . Most important hotkey in Claude Code.

Set a default in `.claude/settings.json` :

```
{ "permissions": { "defaultMode": "acceptEdits" } }
```

Or at startup:

```
claude --permission-mode plan
```

Hook into permission events

Two new hook events let you observe and override decisions deterministically:


```
{
  "hooks": {
    "PermissionRequest": [{ "hooks": [{ "type": "command", "command": "logger -t claude perm-req" }] }],
    "PermissionDenied": [{ "hooks": [{ "type": "command", "command": "~/claude/hooks/notify-denial.sh" }] }],
  }
}
```

Gotcha

- `bypassPermissions` is footgun-grade. Use only inside a container / VM / dev sandbox.
 - `auto` mode looks magical until it isn't — it auto-approves based on a classifier, not policy. Combine it with hard `permissions.deny` rules and a `PreToolUse` hook for production safety.
-

4. CLAUDE.md

Markdown files Claude reads at session start as **persistent instructions**. The system prompt for your project.

Where they live

```
~/ .claude/CLAUDE.md      # YOU – applies to every project
./CLAUDE.md               # PROJECT – committed, shared with team
./CLAUDE.local.md         # LOCAL – gitignored, just you on this repo
.claude/rules/*.md        # Path-scoped rules (frontmatter: paths)
```

Nested `CLAUDE.md` files in subfolders **also load** when Claude reads files in that subfolder. So you can have `frontend/CLAUDE.md` with React conventions and `backend/CLAUDE.md` with Go conventions.

Initialize

```
/init      # auto-generates a CLAUDE.md from your repo
/memory    # view all loaded CLAUDE.md files
```

Imports

```
See @README.md for overview.
Style guide: @.claude/rules/style.md
Personal: @~/ .claude/shared.md
```

Relative paths resolve relative to **the file doing the import**, not your cwd.

Pull CLAUDE.md from extra directories

```
claude --add-dir ../shared-libs
```

```
export CLAUDE_CODE_ADDITIONAL_DIRECTORIES_CLAUDE_MD=1
```

With the env var set, every directory passed via `--add-dir` (or `additionalDirectories` in settings) also contributes its `CLAUDE.md`. Useful for monorepos and shared internal libraries.

What to put in it

✅ Conventions ("use 2-space indent", "all errors return `Result<T, E>`") ✅ Architecture facts ("auth lives in `src/auth/`, never `src/utils/`") ✅ Commands ("run tests with `pnpm test`", "lint with `make lint`") ✅ Don'ts ("never edit `generated/`")

What NOT to put

❌ Secrets ❌ Long reference docs (link with `@import` instead) ❌ Temporary task state (use auto memory or just chat)

Path-scoped rules

```
---
paths:
  - "src/api/**/*.ts"
---

# API Rules
- Validate every input with Zod.
- Return `{ ok, data, error }` envelopes.
```

Gotcha

`CLAUDE.md` is **context**, not enforcement. Claude can ignore it. Be **specific and short** — over 200 lines and adherence drops sharply. Use hooks (§11) when you need a rule that *cannot* be ignored.

5. Auto Memory

The other half of memory: **Claude writes this one**. Your `CLAUDE.md` is a wiki you maintain; auto memory is a notebook Claude keeps.

Where it lives

```
~/ .claude/projects/<hashed-project-path>/memory/  
├─ MEMORY.md           # index – first 200 lines auto-loaded each session  
├─ user_role.md        # who you are  
├─ feedback_*.md       # corrections + validated calls  
├─ project_*.md        # ongoing work, decisions, deadlines  
└─ reference_*.md      # pointers to Linear / Slack / dashboards
```

Toggle

```
/memory           # view, toggle, open folder
```

```
{ "autoMemoryEnabled": true }
```

What Claude saves

- **User memories** — your role, expertise, preferences
- **Feedback memories** — corrections you give ("don't mock the DB"), confirmations ("yes, that PR split was right")
- **Project memories** — deadlines, why-decisions, stakeholder context
- **Reference memories** — "bugs go in Linear project INGEST", "oncall watches dashboard X"

What Claude does NOT save (and shouldn't)

Code patterns (just read the code), git history (just `git log`), debugging recipes (the fix is in the commit), ephemeral task state.

Gotcha

Auto memory can go stale. If a saved memory says "the function `parseUser` lives in `auth.ts`" — verify before acting. The memory is a hint, not truth.

6. The `.claude/` folder

The single source of truth for everything custom about your project.

```
.claude/
├── CLAUDE.md                # project instructions
├── CLAUDE.local.md          # personal notes (gitignored)
├── settings.json            # team-shared config
├── settings.local.json      # personal config (gitignored)
├── agents/
│   └── reviewer.md          # subagent: code reviewer
├── skills/                  # ← live-watched: edit a SKILL.md, no restart needed
│   ├── deploy/
│   │   ├── SKILL.md         # skill: /deploy
│   │   ├── checklist.md     # supporting file
│   │   └── scripts/run.sh   # executable
├── commands/
│   └── pr.md                # custom slash command
├── hooks/
│   └── format-on-edit.sh    # hook script
├── output-styles/
│   └── teaching.md          # custom output style
├── lsp/                    # plugin LSP servers (§13)
├── rules/
│   └── api.md               # path-scoped rules
```

Precedence (highest to lowest)

1. **Managed** — IT-controlled (`/Library/Application Support/ClaudeCode/` , `/etc/claude-code/`)
2. **User** — `~/.claude/`
3. **Project** — `./.claude/`
4. **Local** — `./.claude/settings.local.json`

Higher levels win for conflicts. Tools merge (more permissions = union). Permissions stack (deny wins over allow).

What to commit vs gitignore

Commit: `CLAUDE.md` , `settings.json` , `agents/` , `skills/` , `commands/` , `hooks/` , `rules/` , `output-styles/` , `lsp/` . Gitignore: `CLAUDE.local.md` , `settings.local.json` .

Gotcha

`.claude/skills/` and `.claude/agents/` are **live-watched** — edit a `SKILL.md` mid-session and the change applies to the next invocation, no `/reload-plugins` needed. `settings.json` still requires a session restart.

7. `settings.json`

The control panel. Everything configurable lives here.

Annotated example — copy this and start cutting

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",

  "model": "auto",
  "outputStyle": "Default",
  "includeCoAuthoredBy": true,
  "tui": "default",
  "autoScrollEnabled": true,
  "disableSkillShellExecution": false,

  "permissions": {
    "defaultMode": "acceptEdits",
    "allow": [
      "Bash(npm run *)",
      "Bash(git *)",
      "Bash(pnpm *)",
      "Read(./**)"
    ],
    "deny": [
      "Read(./env*)",
      "Read(./secrets/**)",
      "Bash(curl *)",
      "Bash(rm -rf *)"
    ],
    "ask": [
      "Bash(git push *)",
      "Bash(npm publish *)"
    ],
    "additionalDirectories": ["../shared-libs/"]
  },

  "env": {
    "NODE_ENV": "development",
    "DISABLE_AUTOUPDATER": "0",
    "SLASH_COMMAND_TOOL_CHAR_BUDGET": "8192",
    "ENABLE_PROMPT_CACHING_1H": "1"
  },

  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit",
        "hooks": [
          { "type": "command", "command": "prettier --write \"$CLAUDE_FILE_PATHS\" }
        ]
      }
    ]
  },

  "statusLine": {
    "type": "command",
    "command": "~/claude/statusline.sh"
  }
}
```

```
"enabledPlugins": {
  "python-library-complete@python-library-dev": true
}
```

Notable 2026 settings

Key	What it does
<code>model: "auto"</code>	Anthropic-side router picks Opus/Sonnet/Haiku per turn (Max only)
<code>tui</code>	<code>default</code> , <code>compact</code> , or <code>verbose</code> — terminal layout
<code>autoScrollEnabled</code>	Auto-scrolls to latest output during long tool runs
<code>disableSkillShellExecution</code>	Hard-disables the <code>!`cmd`</code> shell-injection in skills (security)
<code>SLASH_COMMAND_TOOL_CHAR_BUDGET</code>	Per-command description budget; raise it if your skills get truncated
<code>ENABLE_PROMPT_CACHING_1H</code>	Use 1-hour cache TTL instead of 5-min — pays off for long sessions

Permission rule syntax

```
Bash                # all bash
Bash(npm run *)     # only "npm run anything"
Read(~/.zshrc)      # specific file
Edit(./src/**)       # glob inside repo
WebFetch(domain:github.com)
MCP                 # all MCP tools
mcp_github          # one specific MCP server's tools
```

Gotcha

- **Validate JSON** — `jq . .claude/settings.json` . A typo silently disables the file.
- **Protected paths** (`.git/` , `.claude/` , `.mcp.json` , shell rc files) **always prompt** unless you're in `bypassPermissions` .
- `apiKeyHelper` is a script that returns a key — handy for rotating creds, but it runs every session.

8. Slash Commands

Built-in (the ones to memorize)

Type `/` to see every command. Below: the ones worth memorizing, grouped. Bundled **skills** (prompt-driven, can self-invoke) are marked `[Skill]`.

```

GENERAL
/help          show help
/init          generate CLAUDE.md
/clear         reset conversation (keep config)          aliases: /reset, /new
/compact       summarize conversation to save context
/recap         one-line summary of session so far
/usage         cost + plan usage stats                  aliases: /cost, /stats
/model         switch model
/effort        set effort (low/medium/high/xhigh/max) ; /effort auto resets to default
/focus         toggle minimal "last prompt + result" view
/tui           switch renderer (default | fullscreen)
/context       visualize current context usage as a grid
/diff          interactive diff viewer (per-turn + git)
/copy [N]      copy the Nth-latest assistant response

MEMORY & CONFIG
/memory        view + edit CLAUDE.md + auto memory
/permissions   allow / ask / deny rules                alias: /allowed-tools
/config        full config UI                          alias: /settings
/theme         pick a color theme
/keybindings   open keybindings file
/add-dir <path> grant file access to another directory

EXTENSIONS
/agents        manage subagents
/skills        list available skills (press t to sort by token count)
/mcp           manage MCP servers
/hooks         list configured hooks
/plugin        install / manage plugins                alias: /plugins
/reload-plugins pick up plugin changes without restart
/output-style  switch output style
/statusline    configure status line

CHECKPOINTS & SESSIONS
/rewind        checkpoint menu                          aliases: /undo, /checkpoint
/branch [name] branch the conversation                  alias: /fork
/resume [id|name] resume a session                      alias: /continue
/rename        rename current session

REVIEW & SAFETY
/review [PR]   review pending changes (local)
/ultrareview [PR] deep multi-agent review in the cloud
/security-review scan diff for injection, auth, secrets
/fewer-permission-prompts [Skill] auto-tune your allowlist from past sessions
/sandbox       toggle sandbox mode (supported platforms)

WORK ORCHESTRATION
/plan [task]   enter plan mode immediately
/loop [interval] [Skill] run a prompt repeatedly        alias: /proactive
/batch <task>  [Skill] decompose + parallel agents in worktrees
/debug         [Skill] structured debug loop
/simplify      [Skill] review recent changes for reuse / dead code
/claude-api    [Skill] Anthropic SDK / API help
/tasks         list background tasks                    alias: /bashes

CLOUD & SURFACES
/schedule      create / manage routines (§16)           alias: /routines
/desktop       hand session off to Desktop app          alias: /app
/web-setup     connect GitHub to claude.ai/code via gh
/teleport      pull a web session into THIS terminal    alias: /tp
/remote-control expose this session for control from claude.ai alias: /rc
/mobile        QR for mobile app                       aliases: /ios, /android

```

```

/ide          manage IDE integrations
/chrome       configure Claude in Chrome
/autofix-pr   spawn cloud session that watches & fixes the current PR
/ultraplan <task> draft a plan in a cloud session, execute remotely
/install-github-app set up Claude GitHub Actions on a repo
/install-slack-app  install the Claude Slack app
/setup-bedrock  wizard (visible when CLAUDE_CODE_USE_BEDROCK=1)
/setup-vertex  wizard (visible when CLAUDE_CODE_USE_VERTEX=1)

```

DIAGNOSTICS & ACCOUNT

```

/doctor       diagnostics (press f to auto-fix)
/status       version, model, account, connectivity
/insights     analytics on your last 30 days of sessions
/team-onboarding generate a teammate ramp-up doc from your history
/btw <q>      ask a side question without polluting context
/voice        toggle voice dictation (hold | tap | off)
/login        sign in
/logout       sign out
/exit         quit

```

alias: /quit

Custom slash commands

Drop a markdown file in `.claude/commands/` (project) or `~/.claude/commands/` (user):

```

---
description: Open a PR with conventional commit title
argument-hint: "[branch-name]"
allowed-tools: Bash(git *) Bash(gh *)
model: claude-sonnet-4-6
---

```

Open a PR for the current branch into main.

Steps:

1. ``git status`` – verify clean working tree
2. ``git push -u origin HEAD``
3. ``gh pr create --fill --base main``
4. Print the PR URL.

Title prefix: \$ARGUMENTS

Invoke: `/pr feat`

Frontmatter fields

Field	Effect
<code>description</code>	Shows in <code>/</code> menu, helps Claude know when to invoke
<code>argument-hint</code>	UI hint after the slash name
<code>allowed-tools</code>	Pre-approved — no permission prompts
<code>model</code>	Override session model
<code>disable-model-invocation: true</code>	Only the user can fire it (not Claude)

Gotcha

`.claude/commands/` and `.claude/skills/` are **equal-weight** in 2026 — both get indexed, both can auto-invoke. On a name conflict, **the skill wins**. Use commands for one-file quick wins, skills (§9) when you need supporting files, args, lifecycle hooks, or `context: fork`.

9. Skills

Skills are the most important feature in modern Claude Code. Packaged know-how — instructions, supporting files, scripts — that load **only when needed**.

Anatomy

```
.claude/skills/commit/
├─ SKILL.md           # required – the entry point
├─ conventional.md    # optional supporting file
├─ templates/
│   └─ changelog.md
```

SKILL.md example — full 2026 frontmatter

```
---
name: commit
description: Create a conventional commit with proper scope and body.
when_to_use: User says "commit", "ship this", "push it", or asks to create a git commit.
arguments:
  - name: scope
    description: Optional scope override (defaults to inferred from changed files)
allowed_tools: Bash(git *)
model: claude-sonnet-4-6
effort: medium
context: fork
agent: Default
paths: ["**/*"]
shell: true
hooks:
  PostToolUse:
    - matcher: Bash
      command: ./scripts/post-commit-lint.sh
---
```

You are creating a git commit. Follow the conventional commits spec.

Steps:

1. Run ``git status`` and ``git diff --staged``.
2. Pick a type: feat, fix, refactor, docs, test, chore.
3. Pick a scope from the changed files (or use \$scope).
4. Write subject < 60 chars, imperative mood.
5. If breaking change, add ``BREAKING CHANGE:`` footer.
6. Read @conventional.md for the full spec.
7. Inline a status line: `!`git rev-parse --short HEAD`` → previous SHA
8. Run ``git commit -m "...`" (use heredoc for multi-line).`

Print the commit hash when done. Working in: \${CLAUDE_SKILL_DIR}.

Frontmatter fields (2026)

Field	Purpose
<code>name</code>	Slash invocation: <code>/commit</code>
<code>description</code>	Most important field. This is how Claude decides to auto-invoke. Front-load keywords.
<code>when_to_use</code>	Companion to <code>description</code> — natural-language trigger phrases
<code>arguments</code>	Named positional args, addressable as <code>\$(name)</code>
<code>allowed-tools</code>	Pre-approved tools
<code>model</code>	Override session model
<code>effort</code>	<code>low</code> / <code>medium</code> / <code>high</code> / <code>xhigh</code> / <code>max</code> / <code>auto</code>
<code>context: fork</code>	Run skill in a forked context window — doesn't pollute main session
<code>agent</code>	Run as a named subagent (e.g. <code>agent: Explore</code>)
<code>paths</code>	Auto-load only when matching files in context
<code>shell: true</code>	Allow <code>!`cmd`</code> shell injection inside the skill body
<code>hooks</code>	Skill-scoped hooks (auto-cleaned when skill exits)
<code>disable-model-invocation: true</code>	Only user can invoke
<code>user-invocable: false</code>	Only Claude can invoke (background-only skill)

`description` + `when_to_use` are capped at **1,536 characters combined**. Raise per-command budget with `SLASH_COMMAND_TOOL_CHAR_BUDGET` if needed.

Argument substitution

```
$ARGUMENTS      # full string after /skill
$1, $2, ...     # positional
$ARGUMENTS[0]   # zero-indexed
$(name)         # named arg from frontmatter
${CLAUDE_SESSION_ID} # current session UUID
${CLAUDE Effort} # active effort level
${CLAUDE SKILL_DIR} # absolute path to this skill's directory
```

Shell injection (new)

```
Inline output:      !`gh pr list --state open --json number,title`
```

Multi-line block:

```
```\n`git log --oneline -20`\n`git diff --stat HEAD~5`
```

Output is captured into the skill's context. Disable globally with `disableSkillShellExecution: true` in settings.

### Bundled skills (ship in every session)

```
| Skill | What it does |
|-----|-----|
| `/simplify` | Reviews recent changes for reuse / dead code / over-abstraction |
| `/debug` | Structured debugging loop – hypothesize, instrument, verify |
| `/batch` | Run a prompt across multiple files / dirs in parallel |
| `/loop` | Run a prompt on an interval (or self-paced) |
| `/claude-api` | Help with Anthropic SDK / API / prompt caching |
```

These auto-suggest based on `description` matching – you don't need to install them.

### How skills load (progressive disclosure)

- **\*\*Always in context:\*\*** the skill `name` + `description` + `when\_to\_use` (so Claude knows it exists)
- **\*\*Loaded on invocation:\*\*** the body of `SKILL.md`
- **\*\*Loaded on demand:\*\*** supporting files (Claude reads them with the Read tool)

This is why you can ship 50 skills without wrecking your context window.

### Two ways to invoke

```
```\ntext\n/commit # explicit (you ask)\n"alright ship this" # implicit – Claude matches description
```

Skill that runs as a subagent (forked context)

```
---\nname: deep-explore\ndescription: Heavy codebase investigation that shouldn't pollute main context\ncontext: fork\nagent: Explore\neffort: high\n---
```

Investigate everything reachable from the failing test. Return a 200-word summary with `path:line` citations only. Do not propose fixes.

The skill spawns its own context window, runs to completion, and returns just the summary text to the main session. Combine with §10 for parallel research.

Gotcha

- **Description is everything.** `description: "git stuff"` won't trigger. `description: "Create a conventional commit when the user says commit, ship, or push"` will.
 - Skills can live at user level (`~/.claude/skills/`) or project level (`.claude/skills/`). User skills follow you everywhere.
 - Plugin-namespaced skills look like `plugin:skill` — that's fine, treat them the same.
 - **Live-reloaded** — edit a `SKILL.md` mid-session, next invocation picks it up. No restart needed. (`settings.json` still needs a restart.)
 - If your skill description gets silently truncated, raise `SLASH_COMMAND_TOOL_CHAR_BUDGET` .
-

10. Subagents & Agent Teams

A **subagent** is Claude spawning a fresh Claude with a different system prompt, different tools, and **its own context window**.

Why subagents

- **Context hygiene** — exploration agent reads 50 files, returns a 200-word summary. Your main thread stays clean.
- **Specialization** — a “reviewer” subagent has a system prompt for review. A “researcher” has one for search.
- **Parallelism** — fire off 3 in one turn, work in parallel.

Define a subagent

`.claude/agents/explorer.md`:

```
---
name: Explorer
description: Fast read-only investigator. Use for "where is X", "how does Y work", "find all Z" questions.
model: claude-haiku-4-5-20251001
tools: Read Glob Grep WebFetch
skills:
  - simplify
  - debug
hooks:
  Stop:
    - command: echo "explorer done" >> ~/.claude/agent.log
---
```

You are a code explorer. Investigate the codebase and report back.

Rules:

- NEVER edit files or run code that mutates state.
- Use Glob first to find candidate files, then Grep to narrow, then Read.
- Output: a tight summary with ``path:line`` references. Under 300 words.

Frontmatter

Field	Effect
<code>name</code>	Display name in <code>/agents</code>
<code>description</code>	When the parent agent should delegate to it
<code>model</code>	Run cheaper / faster model in subagent
<code>tools</code>	Restrict to a subset (defense + speed)
<code>skills</code>	Pre-load these skills into the subagent's context
<code>hooks</code>	Agent-scoped hooks. <code>Stop</code> here auto-converts to <code>SubagentStop</code> .

Invoke

```
/agents          # interactive picker (you launch)
"Use the Explorer to find ..." # parent Claude delegates via Task tool
```

Agent Teams

When you want multiple specialized subagents on one task, give the parent a roster:

```
.claude/agents/
├── frontend-reviewer.md    # React, ally, Tailwind
├── backend-reviewer.md    # Go, DB indices, error handling
├── infra-reviewer.md      # Terraform, IAM, cost
└── security-reviewer.md   # OWASP, secrets, supply chain
```

```
> Review the open PR. Dispatch each file to the right reviewer in parallel,
> then aggregate findings into one report.
```

The parent Claude reads the diff, fires four subagents in one tool-call batch, waits, merges. The `TeammateIdle` hook fires when a subagent is waiting on the others — useful for cost telemetry.

When to spawn vs when not

✅ Spawn when: you'd be reading >5 files, doing wide search, or running an isolated investigation. ✅
Spawn when: you want a different system prompt (review, plan, refactor). ❌ Don't spawn for: known file paths, single-file edits, follow-up questions.

Gotcha

- Subagents see **only the prompt you give them**. They have no memory of your conversation. Brief them like a smart colleague who just walked in.
- Subagent permissions can only **further restrict** — they can't get more access than the parent.
- If you launch multiple in one message, they run in parallel. Use this.

- `Stop` in a subagent's frontmatter is silently rewritten to `SubagentStop` — don't double-define both.
-

11. Hooks

Hooks are shell commands that fire on events. They give you **deterministic** control — Claude can't ignore them, they're not "vibes."

The events (2026, complete list)

Event	Fires when
Setup	One-time setup at install / first run
SessionStart	Session begins
InstructionsLoaded	After CLAUDE.md + memory finish loading
ConfigChange	settings.json changed mid-session
CwdChanged	Working directory changed
FileChanged	File on disk changed externally
UserPromptSubmit	User hits enter
UserPromptExpansion	Before <code>@file</code> / <code>!cmd</code> substitution
PreToolUse	Before any tool call (can block)
PostToolUse	After tool succeeds
PostToolUseFailure	After tool fails
PostToolBatch	After a parallel tool batch completes
PermissionRequest	About to prompt user for permission
PermissionDenied	User said no
Notification	Status notification
TaskCreated	Internal task list got a new item
TaskCompleted	Internal task marked done
Elicitation	MCP / agent asks user for input
ElicitationResult	User responded to an elicitation
SubagentStart	A subagent launches
SubagentStop	A subagent finishes
TeammateIdle	Subagent waiting on siblings
WorktreeCreate	A worktree is created (e.g. agent isolation)
WorktreeRemove	A worktree is destroyed
Stop	Claude finishes responding
StopFailure	Stop ended with an error
PreCompact	Before <code>/compact</code> runs

Event	Fires when
PostCompact	After <code>/compact</code> finishes
SessionEnd	Session closes

Schema

```
{
  "hooks": {
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "hooks": [
          {
            "type": "command",
            "command": "prettier --write \"$CLAUDE_FILE_PATHS\""
          }
        ]
      }
    ]
  }
}
```

Hook input/output

Hooks receive JSON on stdin (tool name, args, paths). They communicate back via:

- **Exit 0** — allow, continue
- **Exit 1** — non-blocking warning (shown to user)
- **Exit 2** — **block** the tool call, send stderr back to Claude as feedback
- **JSON stdout** — structured response (`{"action": "block", "reason": "..."}`)

Scoped hooks (huge new pattern)

Hooks can live inside a **skill or subagent's frontmatter**. They auto-attach when the skill/agent is active and auto-detach when it exits. No global pollution.

```
----
name: deploy-prod
description: Deploy to production
hooks:
  PreToolUse:
    - matcher: Bash
      command: ~/.claude/hooks/require-main-and-green-ci.sh
  PostToolUse:
    - matcher: Bash
      command: ~/.claude/hooks/post-deploy-slack.sh
----
```

The `require-main-and-green-ci.sh` only fires while the `/deploy-prod` skill is active. Different skills can ship different safety nets without fighting each other.

Five hooks worth stealing

1. Auto-format on edit

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "prettier --write \"$CLAUDE_FILE_PATHS\"
2>/dev/null || true" }]
    }]
  }
}
```

2. Block destructive bash

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Bash",
      "hooks": [{ "type": "command", "command": "~/.claude/hooks/block-rm-rf.sh" }]
    }]
  }
}
```

```
#!/bin/bash
# block-rm-rf.sh
read -r INPUT
if echo "$INPUT" | grep -qE 'rm -rf (/|~|~$HOME)'; then
  echo "Blocked dangerous rm -rf" >&2
  exit 2
fi
exit 0
```

3. Terminal bell when Claude finishes

```
{
  "hooks": {
    "Stop": [{ "hooks": [{ "type": "command", "command": "printf '\\a'" }] }]
  }
}
```

4. Log every prompt to a file

```
{
  "hooks": {
    "UserPromptSubmit": [{
      "hooks": [{ "type": "command", "command": "jq -r '.prompt' >> ~/.claude/prompts.log"
    }]
  }
}
```

5. Auto-run tests after a batch of edits

```
{
  "hooks": {
    "PostToolBatch": [{
      "matcher": "Edit|Write",
      "hooks": [{ "type": "command", "command": "pnpm test --run --silent || (echo 'TESTS FAILED - fix before continuing' >&2; exit 2)" }]
    }]
  }
}
```

`PostToolBatch` is the right hook here — fires once per batch, not once per edit, so a 5-file refactor only runs the suite once.

Gotcha

- Hooks are **synchronous**. A 5-second hook adds 5 seconds to every tool call.
- Use `disableAllHooks: true` in settings to nuke them temporarily for debugging.
- `PreToolUse` exit-2 is the most powerful pattern: **block + explain** — Claude reads the stderr and self-corrects.
- Scoped hooks (skill/agent frontmatter) win over global ones if both match — skill scope is more specific.

12. MCP

Model Context Protocol = standardized way for external systems to expose tools to Claude.

GitHub, Slack, Postgres, Linear, Notion, Puppeteer, your internal API — they all become tools Claude can call.

Install an MCP server

```
claude mcp add # interactive
```

Or hand-edit `.mcp.json` :

```
{
  "mcpServers": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_..." }
    },
    "postgres": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres", "postgres://..."]
    },
    "puppeteer": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-puppeteer"]
    }
  }
}
```

Transports

Type	When to use
<code>stdio</code>	Local process. Default. Easiest.
<code>http</code>	Remote HTTP server (e.g., a corporate gateway)
<code>sse</code>	Server-Sent Events for streaming

Scopes

```
~/mcp.json          # follows you
.mcp.json           # committed, team-shared
.mcp.local.json     # personal overrides, gitignored
```

Use it

```
/mcp                # menu of servers + their tools
@github:pr/123      # reference an MCP resource in a prompt
/mcp__github__create_issue # invoke an MCP-exposed prompt
```

Runtime toggling

```
/mcp toggle github    # disable for this session
/mcp toggle github on  # re-enable
```

Useful when a slow MCP server is blowing up session start — disable it, do your work, flip it back.

Human-in-loop with elicitation

When an MCP tool needs user input mid-call, it raises an **elicitation**. Two hooks let you intercept:

```
{
  "hooks": {
    "Elicitation": [{ "hooks": [{ "type": "command", "command": "logger -t mcp
                           elicit" }] }],
    "ElicitationResult": [{ "hooks": [{ "type": "command", "command":
                           "~/claude/hooks/audit-elicat.sh" }] }]
  }
}
```

Use this for compliance trails (“every time the prod-DB MCP server asks for confirmation, log who said yes”).

The MCP servers worth installing first

- **GitHub** — PRs, issues, comments
- **Filesystem** — controlled access to other folders
- **Puppeteer** — browser automation, screenshots, end-to-end smoke tests
- **Postgres / SQLite** — query your DB during dev
- **Slack** — post status, read channels
- **Sentry** — pull errors into context
- **AWS Docs** — pull official AWS docs into context (no fake URLs)

Gotcha

- Slow MCP servers slow down session start. Disable ones you don't use today (`/mcp toggle <name>`).
 - OAuth servers open a browser the first time. Have a port free (default 8080).
 - Tool name conflicts are namespaced as `mcp__server__tool` — you'll see them in `/permissions` .
 - If an MCP server appears in multiple scopes (`~/mcp.json` + `.mcp.json` + `.mcp.local.json`), local wins. Run `claude doctor` if you're seeing duplicates.
-

13. Plugins & Marketplaces

Plugins are **bundles** — a single install ships skills, agents, hooks, commands, output styles, MCP servers, and **LSP servers** together.

Install

```
/plugin # interactive
```

Or via settings:

```
{
  "enabledPlugins": {
    "python-library-complete@python-library-dev": true
  },
  "extraKnownMarketplaces": {
    "trainwithshubham": {
      "source": { "source": "github", "repo": "shubham-londhe/claude-plugins" }
    }
  }
}
```

Build your own plugin

```
my-plugin/
├─ plugin.json
├─ skills/deploy/SKILL.md
├─ agents/reviewer.md
├─ commands/release.md
├─ hooks/format.sh
├─ output-styles/teaching.md
├─ mcp-servers.json
└─ lsp/
   └─ my-lang-server.json    # language server for diagnostics + go-to-def
```

The `lsp/` directory is new in 2026 — plugins can ship Language Server Protocol servers that Claude uses for `LSP` tool calls (real diagnostics, not `grep`).

Push to GitHub, point a marketplace at it, share the install link. Done.

Gotcha

After editing a plugin source you control, run `/reload-plugins` to pick up changes without restarting the session. New skills/agents inside the plugin live-reload; new hooks and LSP servers don't.

14. IDE Integrations & the 5 Surfaces

The same `claude` engine runs in 5 places. Settings sync across them via `~/.claude/`.

Terminal — the default

Just `claude`. Power-user mode. Everything in this guide works here.

VS Code

```
code --install-extension Anthropic.claude-code
```

- `/` menu in editor
- Selection becomes context automatically
- Inline diff view for edits
- Session sidebar

JetBrains (IntelliJ, PyCharm, WebStorm...)

Settings → Plugins → Marketplace → search "Claude Code" → install → restart.

Desktop app

Download from `claude.ai/download` or hand off from the terminal:

```
/desktop # current session jumps to the Desktop app
```

The Desktop app is best for multi-session workflows — you can have 4 sessions open in tabs, each in a different repo.

Web — `claude.ai/code`

Zero-install. Runs in a browser sandbox. Routines (§16) live here. Link a repo:

```
/web-setup # registers this repo with claude.ai/code
```

Mobile (Claude mobile app)

- `/mobile` (or `/ios` / `/android`) shows a QR to install.
- **Remote Control** (`/remote-control`, alias `/rc`) — make this terminal session controllable from `claude.ai` on your phone. Send prompts, watch tool calls, approve permissions remotely.

Slack

```
/install-slack-app          # OAuth flow opens in browser
```

Mention `@Claude` in a thread; it picks up the conversation context and can open a PR back.

Cross-surface

```
/teleport          # pull a web session into THIS terminal (alias /tp)
/desktop           # push terminal session to Desktop app (alias /app)
/web-setup         # connect this terminal's GitHub to claude.ai/code
/ide               # manage IDE integrations from terminal
```

Gotcha

The VS Code, JetBrains, and Desktop integrations **run the same `claude` CLI** under the hood. Settings (`~/.claude/`) are shared. One mental model. Web has its own sandbox — repos must be mounted via the cloud environment, not your local FS.

15. Headless Mode + Agent SDK

Run Claude Code in **scripts, CI, cron jobs, GitHub Actions, GitLab CI/CD**.

-p flag (one-shot prompts)

```
claude -p "Summarize what changed in this branch"

# Auto-approve specific tools
claude -p "Fix failing tests" --allowedTools "Read,Edit,Bash(pnpm test)"

# JSON output for parsing
claude -p "List all TODO comments" --output-format json

# Stream tokens
claude -p "Refactor auth.ts" --output-format stream-json --verbose | jq -r
      '.event.delta.text?'

# Continue last session
claude -p "Now write tests for it" --continue

# Fork a session (Checkpoints – see §17)
claude --continue --fork-session

# Resume a specific session
claude -p "Pick up where we left off" --resume <session-id>

# Permission mode
claude -p "Deploy" --permission-mode acceptEdits

# Bare mode – skip auto-loading skills/MCP/hooks (reproducible CI)
claude --bare -p "Run lint" --allowedTools Bash

# Cap CI cost
claude -p "Refactor this module" --max-budget-usd 2.50

# Structured output (validates against your schema)
claude -p "Extract issues from log.txt" \
      --json-schema '{"type":"object","properties":{"issues":{"type":"array"}}}'

# Resume a session linked to a PR (GitHub / GitLab MR / Bitbucket PR)
claude --from-pr 482

# Don't persist the session (single-shot, no transcript on disk)
claude -p "..." --no-session-persistence
```

Useful subcommands (not slash, but `claude <sub>`)

```
claude install latest      # pin update channel
claude doctor              # health check
claude project purge .    # wipe all Claude state for a project
claude auto-mode          # introspect the auto-mode classifier
claude ultrareview        # cloud multi-agent review, non-interactive
claude mcp add             # register an MCP server interactively
claude plugin install <name> # install + enable a plugin
claude setup-token        # mint a long-lived auth token
```

Official integrations

- **GitHub Actions** — `anthropics/claude-code-action@v1`
- **GitHub Code Review** — auto-review on `pull_request.opened`
- **GitLab CI/CD** — official template available
- **Slack** — `@Claude` in a thread, get a PR back

CI example — auto-fix lint on PR

```
# .github/workflows/auto-lint.yml
- run: |
  npm install -g @anthropic-ai/claude-code
  claude --bare -p "Fix all eslint errors in this PR" \
    --allowedTools "Read,Edit,Bash(pnpm lint --fix)" \
    --permission-mode acceptEdits
  git diff --quiet || (git commit -am "auto-fix: lint" && git push)
env:
  ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
```

Enterprise backends

```
export CLAUDE_CODE_USE_BEDROCK=1    # AWS Bedrock
export CLAUDE_CODE_USE_VERTEX=1    # Google Vertex AI
export CLAUDE_CODE_USE_FOUNDRY=1    # Microsoft Foundry (2026)
```

Or interactive: `/setup-bedrock` , `/setup-vertex` . Three providers, same `claude` binary, traffic stays in your cloud.

Agent SDK (programmatic)

Use the SDK over `-p` when you need to: - Maintain state across many calls in one process - Drive tool-use loops yourself (e.g. inject a synthetic user response) - Stream into a custom UI

Python:

```
pip install claude-agent-sdk
```



```
from claude_agent_sdk import query

async for msg in query(prompt="Find and fix the off-by-one bug in pagination.py"):
    print(msg)
```

TypeScript:

```
npm install @anthropic-ai/claude-agent-sdk
```

```
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const msg of query({ prompt: "Refactor auth.ts to use JWT" })) {
    console.log(msg);
}
```

Gotcha

- Headless runs **inherit your local config** unless you pass `--bare`. Use `--bare` in CI.
 - Non-interactive runs **abort on permission denial** — there's no human to approve. Pre-approve everything via `--allowedTools` or `permissions.allow`.
 - For enterprise, set `ENABLE_PROMPT_CACHING_1H=1` in CI — long sessions amortize cache misses.
-

16. Routines

Routines = Claude Code running on Anthropic's cloud, on a schedule or trigger, while your laptop is closed. This is the single biggest 2026 feature.

A routine has three parts: 1. **Prompt** — the task ("triage open issues, label by component, comment on stale ones") 2. **Trigger** — what fires it (cron / API / GitHub event) 3. **Surface** — where it runs (Anthropic's cloud, with optional repo mount)

Create one

```
/schedule  
> "every weekday at 9am, review the open PRs in shubham-londhe/claude-plugins, comment on  
    stale ones, label by area"
```

Claude drafts the routine, asks for confirmation, registers it.

```
/schedule list           # all your routines  
/schedule update <id>   # edit  
/schedule run <id>      # one-off invoke now  
/schedule in 2 weeks, ... # natural-language one-off
```

Three trigger types

Trigger	When it fires	Min interval
Schedule	Cron expression	1 hour
API	HTTPS webhook to a per-routine endpoint	none
GitHub	<code>pull_request.*</code> , <code>release.*</code> , etc.	none

Schedule trigger

```
> /schedule cron "0 9 * * 1-5" - review open issues, label triage
```

API trigger

```
curl -X POST https://api.anthropic.com/v1/claude_code/routines/<id>/fire \  
  -H "Authorization: Bearer $ROUTINE_TOKEN" \  
  -H "anthropic-beta: experimental-cc-routine-2026-04-01" \  
  -H "anthropic-version: 2023-06-01" \  
  -H "Content-Type: application/json" \  
  -d '{"text": "Sentry alert SEN-4521 fired in prod. Stack trace attached."}'
```

Per-routine bearer token (generate from the routine's edit page on the web; shown once, not retrievable). Beta header required during research preview. Body accepts an optional `text` field — freeform string, passed alongside the saved prompt.

A successful POST returns the new session URL:

```
{
  "type": "routine_fire",
  "claude_code_session_id": "session_01HJKLMN0PQRSTUVWXYZ",
  "claude_code_session_url": "https://claude.ai/code/session_01HJKLMN0PQRSTUVWXYZ"
}
```

GitHub trigger

```
# pinned in routine config UI
trigger:
  type: github
  events: [pull_request.opened, pull_request.synchronize]
  filters:
    base_ref: main
    paths: ["src/**"]
```

Usage and limits

Routines draw down your subscription usage like interactive sessions. **Each account has a daily cap on routine runs that varies by plan tier** — see your current allowance and consumption at claude.ai/code/routines or claude.ai/settings/usage. One-off runs (`/schedule in 2 weeks, ...`) are **exempt** from the daily cap. Organizations with extra usage enabled keep running on metered overage past the cap.

A real routine

```
NAME:    nightly-flake-triage
TRIGGER: cron "0 3 * * *" (3am UTC)
PROMPT:
  1. Run `pnpm test --run --reporter=json` and find tests that pass on retry.
  2. For each flake, open a GitHub issue with the failure log.
  3. If a flake has 3+ issues already, post in #eng-quality on Slack via the Slack MCP.
  4. Stop after 30 min wallclock.
ALLOWED TOOLS: Read, Bash(pnpm *), Bash(gh *), mcp_slack__post
```

By 9am you have a triage report and your laptop never opened.

Gotcha

- Routines **run as you** — they push to GitHub with your identity, post to Slack as you. Treat the routine token like a deploy key.
- By default, routines can only push to `claude/` -prefixed branches. Toggle “Allow unrestricted branch pushes” per-repo if you need writes to other branches.

- Routines run in Anthropic's cloud, not your laptop. Each repo is cloned at run start; configure cloud env (network, env vars, setup script) before saving.
 - 1-hour minimum on cron is a hard floor during research preview. Use API triggers for sub-hour cadence.
 - The `/fire` API surface is in research preview behind `experimental-cc-routine-2026-04-01`. Breaking changes ship behind new dated headers; the two prior versions stay live so callers can migrate.
 - A routine that fails silently can chew the cap fast — wire `Stop` + `StopFailure` hooks for telemetry.
-

17. Checkpoints

Checkpoints = git for your AI session. Every Edit and Write is automatically snapshot. You can roll back the conversation, the code, or both — for 30 days.

Open the menu

```
Esc Esc      # checkpoint picker
/rewind      # same thing
/undo        # alias
```

You see a list of checkpoints with timestamps + the prompt that led to each. Pick one.

Four restore actions

Action	Code	Conversation
Restore code + conversation	rolled back	rolled back
Restore conversation only	kept	rolled back
Restore code only	rolled back	kept
Summarize from here	kept	replaced with summary

The last one is the killer feature — keep your code state, replace 200 messages of back-and-forth with a tight summary, keep going.

Persistence

- Checkpoints persist **across sessions for 30 days**.
- Tracked: `Edit`, `Write`, `NotebookEdit` tool calls.
- **Not tracked:** bash mutations (`rm`, `mv`, `git reset --hard`, anything in `Bash`). If a routine `rm -rf` 'd your `node_modules`, the checkpoint won't bring it back.

Fork a session

```
claude --continue --fork-session
```

Picks up the last session but branches it — the original timeline stays intact, you explore from the fork. Useful for “what if” rewrites.

Compose with `/recap`

```
> /recap  
[1-paragraph summary of this session]  
  
> Esc Esc  
[pick the checkpoint at "before the failed refactor"]  
[choose: Summarize from here]
```

Now you have the right code state + a clean conversation seeded with a recap. Best context surgery in the tool.

Gotcha

- Bash mutations aren't tracked. If your edit *plus* a bash side-effect went wrong, the rewind only undoes the edit. Keep destructive bash gated by hooks (§11).
 - 30-day TTL is per session, not global. Resuming a 31-day-old session works, but its checkpoints are gone.
 - `/compact` and `Esc Esc → Summarize from here` both shorten context, but compact rewrites the whole transcript while summarize is anchored to a checkpoint. Prefer summarize when you know *where* the noise started.
-

18. Output Styles

Output styles change Claude's **tone and format**, not its capabilities.

Built-in

- **Default** — software engineering focus, terse
- **Explanatory** — adds educational asides
- **Learning** — pair programming mode, has you write some code

Switch

```
/output-style          # picker
```

```
{ "outputStyle": "Explanatory" }
```

Custom — `~/.claude/output-styles/tutor.md`

```
----  
name: Tutor  
description: Step-by-step teaching mode  
keep-coding-instructions: true  
----
```

When helping with code:

1. State the concept in one sentence.
2. Show a minimal example.
3. Ask the user to predict the output.
4. Reveal and explain.
5. Suggest one variation to try.

Always end with: "Want to go deeper?"

Gotcha

Output styles take effect on the **next session** (system prompt is fixed for the duration of a session for prompt caching).

19. Hidden Gems

The stuff that makes you look like a wizard.

Bash escape

```
! pnpm test
! gh pr list --state open
```

Run a command, get output into context — without burning a Bash tool call.

Quick memory dump

```
# never use eslint-disable in this repo
```

Single line, prefix with `#`, and it's appended to `CLAUDE.md` immediately.

File reference in prompt

```
@src/auth.ts is broken when the token has special chars
```

Effort dial

```
/effort                                # arrow-key slider: low / medium / high / xhigh / max
/effort auto                           # reset to model default
```

`xhigh` and `max` are Opus 4.7-only and Max-plan-only. `max` is session-only (doesn't persist). Use `${CLAUDE_EFFORT}` inside skills to adapt instructions to the active level.

Verbose transcript

```
Ctrl+O                                # toggle: see raw tool inputs/outputs
```

Live-debug what Claude is actually sending to tools.

`--continue` and `--resume`

```
claude --continue                      # last session in this folder
claude --resume                        # picker for any past session
claude --continue --fork-session      # branch the last session
```


Plan mode for big changes

```
Shift+Tab          # cycle to plan mode
"Refactor the entire auth system to use JWT"
# Claude drafts a plan. You approve. Then it runs.
```

Background a long task

Ctrl+B while a tool is running. Claude continues, you can keep chatting. Output streams to a temp file.

Vim mode

```
/config → Editor → Vim
```

Real Vim bindings in the prompt buffer.

Custom status line

```
/statusline
```

Or hand-write `~/.claude/statusline.sh` :

```
#!/bin/bash
INPUT=$(cat)
MODEL=$(echo "$INPUT" | jq -r '.model.display_name')
DIR=$(echo "$INPUT" | jq -r '.workspace.current_dir' | xargs basename)
EFFORT=$(echo "$INPUT" | jq -r '.effort // "auto"')
echo "$MODEL $DIR effort:$EFFORT"
```

Switch model mid-session

```
/model
```

Cross-surface tricks

```
/teleport          # pull a web session into THIS terminal (alias /tp)
/desktop           # push current session to Desktop app (alias /app)
/web-setup          # connect this terminal's GitHub to claude.ai/code
/remote-control    # let your phone drive this session (alias /rc)
```

Worktrees + tmux for parallel sessions

```
claude -w feat/auth      # spawn a session in a fresh git worktree
claude -w feat/auth --tmux # also create a tmux session (uses iTerm panes if available)
```

The `/batch` skill uses this internally — decomposes a task into 5–30 units and spawns one isolated agent per worktree.

Branch the conversation, not just the code

```
/branch trying-jwt          # forks the current conversation, alias /fork
/resume                     # jump back to the original later
```

Different from `/rewind` (which deletes future) — `/branch` keeps both timelines.

`--from-pr` to resume from a PR

```
claude --from-pr 482          # GitHub
claude --from-pr https://gitlab.com/.../merge_requests/12
```

Picks up the session linked to that PR, so a CI agent and a human reviewer share state.

Side question without polluting context

```
/btw what's the difference between proxy_pass and rewrite in nginx?
```

Asks once, gets the answer, doesn't add to your main conversation history.

Visualize context usage

```
/context                    # colored grid of what's eating your tokens
/insights                   # 30-day analytics on your sessions
```

Voice dictation

```
/voice hold                 # hold-to-talk
/voice tap                  # tap-to-toggle
```

`--dangerously-skip-permissions`

The legendary flag. Skip all permission prompts. **Use only in containers.** Equivalent to `bypassPermissions` mode.

Tool search

If your account has many tools (lots of MCP), Claude defers loading rare tools and uses `ToolSearch` to find them on demand. You don't need to do anything — just know that's why some tools "appear" mid-session.

Long cache TTL

```
export ENABLE_PROMPT_CACHING_1H=1
```

5-min cache → 1-hour cache. Pays for itself if your session lasts >10 min between turns.

20. Pro Patterns

How the pieces compose. The part most tutorials skip.

Pattern 1 — The “research → plan → execute” stack

1. **Subagent** (Explorer): reads 30 files, returns 200-word summary.
2. **Plan mode**: Claude drafts the change.
3. **You approve**.
4. **Skill** (`/commit`): conventional commit at the end.
5. **Hook** (`PostToolUse Edit`): prettier auto-formats every edit along the way.

Pattern 2 — Auto-fix loop in CI

- GitHub Action runs `claude --bare -p "Fix lint errors" --allowedTools ...`
- Hook runs `pnpm lint` after every Edit
- If lint fails (exit 2), Claude reads the error and tries again
- Loop terminates when lint is clean

Pattern 3 — Per-stack subagent team

```
.claude/agents/  
├─ frontend-reviewer.md      # React, ally, Tailwind  
├─ backend-reviewer.md      # Go, DB indices, error handling  
└─ infra-reviewer.md        # Terraform, IAM, cost
```

You: “Review this PR.” Claude reads the diff, dispatches each file to the right reviewer **in parallel**, aggregates findings.

Pattern 4 — Skill that calls a subagent

```
---  
name: deep-debug  
description: Hard bug investigation  
context: fork  
---  
  
1. Spawn the Explorer subagent to map all callers of the failing function.  
2. Spawn the LogReader subagent to pull last 200 lines of relevant logs.  
3. Synthesize a hypothesis.  
4. Propose a fix.
```

Pattern 5 — Hook + skill safety net

- Skill `/deploy-prod` — fires deploy.

- Scoped hook in the skill's frontmatter — blocks `Bash` if branch isn't `main` and tests aren't green.

You can't deploy junk by accident. Determinism + intelligence.

Pattern 6 — MCP-driven dev loop

- Postgres MCP: Claude queries the DB to verify schema.
- Puppeteer MCP: Claude opens the browser, takes a screenshot, verifies the UI.
- GitHub MCP: Claude opens the PR with screenshot attached.

A full feature, end-to-end, no human between "build it" and "PR up."

Pattern 7 — Routine-driven autonomy

Three routines that run while you sleep:

nightly-flake-triage	cron "0 3 * * *"	opens issues for flaky tests
pr-incoming-review	github pull_request.*	posts review comments
deploy-verify	api trigger	curl'd from your CD pipeline

By morning your inbox has issues filed, PRs reviewed, and deploys verified — all attributed to you, all rolled back via `/rewind` if something went sideways.

Pattern 8 — Self-improving session

```
{
  "hooks": {
    "Stop": [{
      "hooks": [{ "type": "command", "command": "~/.claude/hooks/refine-claude-md.sh" }]
    }]
  }
}
```

The hook reads the session transcript, finds places where you corrected Claude, and proposes additions to `CLAUDE.md`. Over weeks, the project's instructions get sharper without you maintaining them.

21. Troubleshooting

Symptom	Fix
Claude ignores <code>CLAUDE.md</code>	<code>/memory</code> to verify it loaded; rewrite vague rules to specific ones
Skill never auto-triggers	Rewrite <code>description</code> + <code>when_to_use</code> with the exact words a user would say
Skill description truncated	Raise <code>SLASH_COMMAND_TOOL_CHAR_BUDGET</code> in env
<code>settings.json</code> has no effect	<code>jq . .claude/settings.json</code> to validate JSON; check precedence
Skill edit didn't pick up	Skills live-reload — but <code>settings.json</code> and hooks need <code>/reload-plugins</code> or restart
MCP server not appearing	<code>/mcp</code> for status; check <code>.mcp.json</code> syntax; if duplicated across scopes, run <code>claude doctor</code>
Endless permission prompts	<code>/fewer-permission-prompts</code> auto-tunes your allowlist; or switch to <code>acceptEdits</code> / <code>auto</code> mode
Hook not firing	Run <code>/hooks</code> to verify it's registered; check matcher regex; for skill-scoped hooks, the skill must be active
Session lost context	<code>/recap</code> for a quick summary; <code>Esc Esc → Summarize from here</code> for surgery; <code>/compact</code> as last resort
"Tool not found"	Tool may be deferred — Claude will search and load it via ToolSearch
Routine didn't fire	Check the daily cap at claude.ai/code/routines ; for GitHub triggers, verify the Claude GitHub App is installed (separate from <code>/web-setup</code>); confirm the trigger filters match the actual event
Checkpoint missing	30-day TTL; bash mutations aren't tracked — only

Symptom	Fix
	Edit/Write/NotebookEdit

When all else fails:

```
claude doctor
```

22. Cheat Sheet

INSTALL

<code>curl -fsSL https://claude.ai/install.sh bash</code>	macOS / Linux / WSL
<code>irm https://claude.ai/install.ps1 iex</code>	Windows PowerShell
<code>brew install --cask claude-code</code>	Homebrew
<code>npm install -g @anthropic-ai/claude-code</code>	npm
<code>claude install latest</code>	pin update channel
<code>claude doctor</code>	
<code>/login</code>	

LIVE KEYS

<code>Shift+Tab</code>	cycle permission mode (default → acceptEdits → plan → auto)
<code>Esc Esc</code>	/rewind – checkpoint menu (4 restore modes)
<code>Ctrl+B</code>	background current task
<code>Ctrl+O</code>	verbose transcript (raw tool I/O)
<code>! cmd</code>	run bash, capture output
<code># text</code>	save to CLAUDE.md
<code>@file</code>	inject file
<code>/</code>	slash menu

FILES YOU OWN

<code>CLAUDE.md</code>	project instructions
<code>~/.claude/CLAUDE.md</code>	personal instructions
<code>.claude/settings.json</code>	project config (live-reloads on /reload-plugins)
<code>.claude/settings.local.json</code>	personal config (gitignored)
<code>.claude/agents/*.md</code>	subagents (live-reload)
<code>.claude/skills/<name>/SKILL.md</code>	skills (live-reload)
<code>.claude/commands/*.md</code>	custom slash commands (equal-weight to skills)
<code>.claude/hooks/*.sh</code>	hook scripts
<code>.claude/output-styles/*.md</code>	output styles
<code>.claude/lsp/*.json</code>	language servers (plugins)
<code>.mcp.json</code>	MCP servers

MUST-KNOW COMMANDS

<code>/init</code>	generate CLAUDE.md	<code>/usage</code>	cost + plan stats
<code>/memory</code>	view memory	<code>/context</code>	visualize tokens
<code>/agents</code>	pick subagent	<code>/skills</code>	list skills
<code>/mcp</code>	manage MCP	<code>/diff</code>	interactive diff
<code>/permissions</code>	permission rules	<code>/effort</code>	thinking dial
<code>/review</code>	review pending changes	<code>/focus</code>	minimal view
<code>/ultrareview</code>	cloud multi-agent review	<code>/tui</code>	switch renderer
<code>/security-review</code>	scan diff for vulns	<code>/recap</code>	session summary
<code>/rewind /undo</code>	checkpoint menu	<code>/branch /fork</code>	branch the convo
<code>/schedule</code>	create routines	<code>/resume</code>	past sessions
<code>/reload-plugins</code>	pick up plugin changes	<code>/desktop</code>	hand off
<code>/web-setup</code>	connect GitHub to web	<code>/teleport /tp</code>	pull web → terminal
<code>/remote-control</code>	drive from phone	<code>/voice</code>	dictation
<code>/loop /proactive</code>	recurring prompt	<code>/batch</code>	parallel agents
<code>/simplify</code>	review recent changes	<code>/debug</code>	structured debug
<code>/claude-api</code>	Anthropic SDK help	<code>/btw</code>	side question

HEADLESS

<code>claude -p "..."</code>	<code>--allowedTools "..."</code>	<code>--output-format json</code>
<code>claude --bare -p "..."</code>	clean CI mode (no skills/MCP/hooks)	
<code>claude --continue</code>	<code>--fork-session</code>	branch a session
<code>claude -p "..."</code>	<code>--max-budget-usd 2.5</code>	cost cap


```
claude -p "... " --json-schema '...' structured output
claude --from-pr 482 resume session linked to a PR
claude -w feat/x --tmux worktree + tmux for parallel sessions
claude install latest pin update channel
claude project purge wipe all project state
claude auto-mode introspect classifier
claude ultrareview cloud review, non-interactive
```

ROUTINES (cloud, while-you-sleep)

```
/schedule cron "0 9 * * 1-5" - task recurring
/schedule in 2 weeks, ... - task one-off (exempt from daily cap)
GitHub triggers configured on the web at claude.ai/code/routines
curl POST /v1/claude_code/routines/<id>/fire (API trigger, beta header required)
Daily cap varies by plan - see claude.ai/code/routines
```

CHECKPOINTS (in-session + 30-day)

```
Esc Esc menu aliases: /rewind /undo /checkpoint
4 modes: code+conv / conv only / code only / summarize-from-here
Tracks Edit/Write/NotebookEdit only - Bash mutations NOT tracked
```

ENTERPRISE BACKENDS

```
CLAUDE_CODE_USE_BEDROCK=1 then /setup-bedrock
CLAUDE_CODE_USE_VERTEX=1 then /setup-vertex
Foundry available as a 3P provider via --bare config
```

POWER MOVES

```
Shift+Tab → plan mode → big refactor
Skill + scoped hook = self-contained safety net
Subagent team = parallel review
MCP = anything-as-a-tool
Routine = autonomy while you sleep
Checkpoint = git for AI sessions
Worktree + /batch = 30 parallel agents on one task
Plugin = ship all the above to your team
```

Claude Code In One Shot · TrainWithShubham · 2026 Edition
trainwithshubham.com · youtube.com/@TrainWithShubham

Outro — what to build next

Three projects to lock in everything in this video:

1. **The Personal Skill Pack.** Build 5 skills you'll use weekly: `/commit` , `/pr` , `/review` , `/test` , `/explain` . Push them to `~/.claude/skills/` . Now they follow you to every project.
2. **The Repo Bootstrap.** A `.claude/` folder you drop into every new repo: `CLAUDE.md` + `settings.json` + 2 hooks (auto-format, block destructive bash) + 1 subagent (reviewer) + 1 routine (nightly triage). Make it a git template repo.
3. **The Custom Agent.** Pick one workflow you do every day. Build a skill that runs it, a scoped hook that protects it, a subagent that researches for it, and a routine that fires it on schedule. Now you've built your own coding agent on top of Claude Code.

Ship it.

About TrainWithShubham

TrainWithShubham is a YouTube channel + community for **DevOps, SRE, Cloud, and Platform engineers** leveling up with AI. Shubham Londhe (the human behind it) works at **Temporal** and runs the channel on the side — the focus is practical, no-fluff, “build something today” content.

If this guide helped:

- **Visit** → trainwithshubham.com — courses, blog, community
- **Subscribe** → youtube.com/@TrainWithShubham — new videos every week
- **Connect** → [Shubham on LinkedIn](#) · [@trainwithshubham on X](#)
- **Share** the video link with one engineer who’s still copy-pasting from a chat window

Want this guide updated?

It will go stale — Claude Code ships fast. The repo includes `.claude/agents/course-updater.md`, a subagent that researches the latest YouTube videos + Anthropic changelog and proposes edits. Run it monthly:

```
> use CourseUpdater to find what's new in Claude Code and propose updates
```

Citation

```
TrainWithShubham. (2026). Claude Code in One Shot – The Complete Guide (2026 Edition).  
Retrieved from https://www.youtube.com/@TrainWithShubham
```

Free to share, fork, translate, and remix. Attribute TrainWithShubham when you do.

```
End of guide. 2026 edition. Go cook.
```

```
– Shubham Londhe  
trainwithshubham.com · @trainwithshubham
```